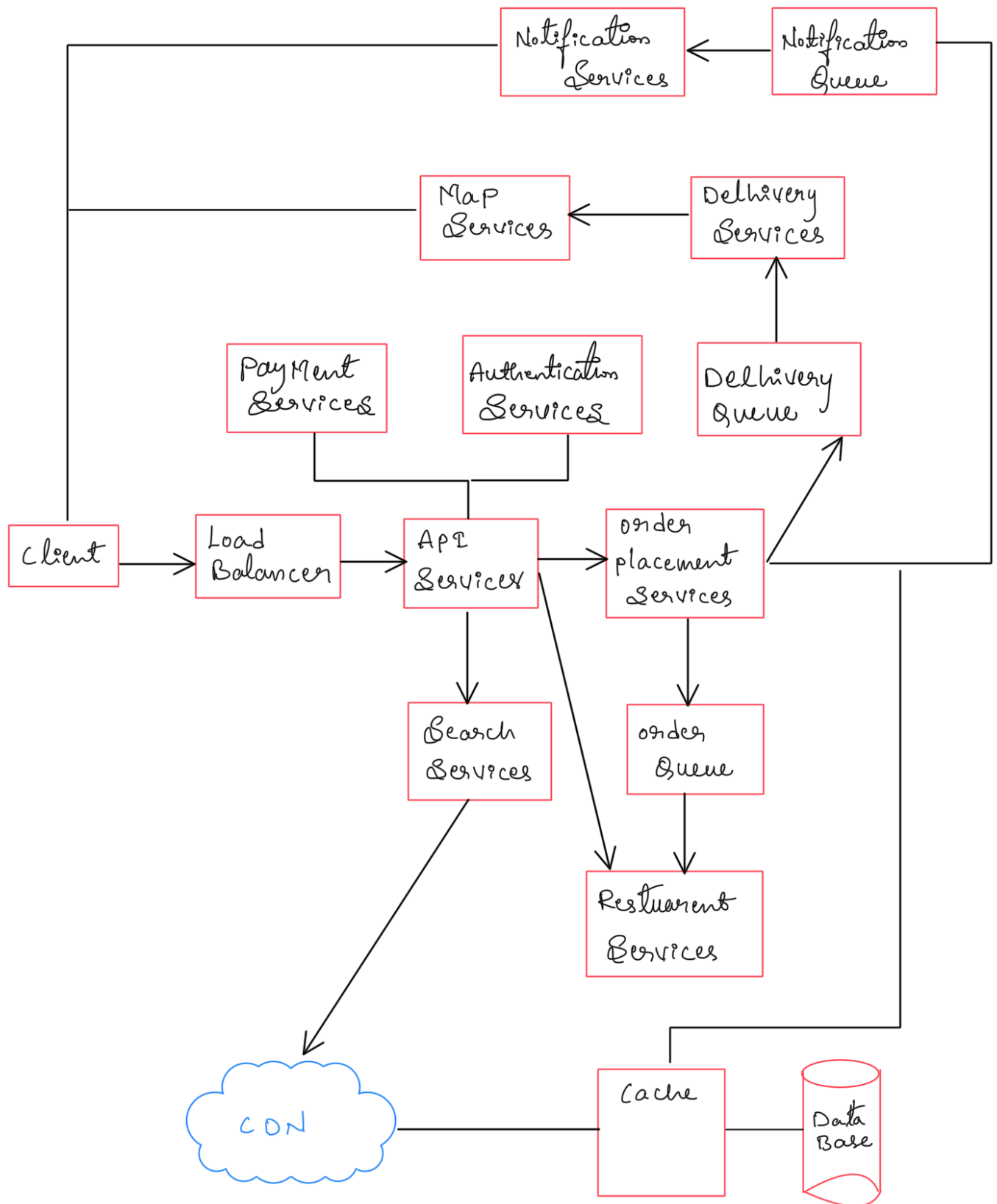


Microservices Used In Restaurant Management System



Low Level Design of Restaurant Management System

1. Client

Customers initiate requests to the load balancer for various services, such as user registration, restaurant searches, and order placement. Additionally, delivery agents and restaurant admins utilize the load balancer to send requests for registration or authentication.

2. [Load Balancer](#)

This vital component monitors server health, rerouting traffic in the event of failures to ensure minimal downtime. Not only does it enhance security by mitigating DDoS attacks, but it also plays a crucial role in scaling applications. Available in hardware, software, and cloud-based forms, load balancers act as traffic managers, directing the flow in modern IT infrastructures.

3. API Services

Serving two key functions, API services handle user authentication, ensuring only authorized users access the system. Once authenticated, these services act as intermediaries for communication with various microservices such as order placement, payment, and search. For example, implementing Kafka for message queuing ensures reliable communication between microservices.

4. Authentication Services

Offering robust user authentication and user registration functionalities, these services are pivotal for security and onboarding new users. The registration process assigns a unique user ID and securely stores essential user information, including names, addresses, and payment details, within the system's database.

5. Payment Services

Offering a variety of payment options, Payment Services ensure flexibility and convenience for settling bills. They validate payments, confirming transaction accuracy and the availability of funds. This process prevents errors, fraud, or unsuccessful payments, ensuring a smooth and trustworthy payment experience. For instance, integrating with a payment gateway like Stripe or PayPal enhances transaction security.

6. [Content Delivery Network\(CDN\)](#)

Specializing in storing static assets like dish images and menus, CDNs reduce the load on the main server, enhancing content accessibility. Positioned strategically, CDNs ensure swift data retrieval, minimizing latency, and speeding up content delivery. For example, using Akamai or Amazon CloudFront for CDN services optimizes the loading time of menu pages, improving the overall user experience.

7. Restaurant Search Services

Enabling users to search for restaurants based on criteria like ratings and cuisines, these services leverage ElasticSearch for rapid data retrieval in a distributed database. For example, users can easily find top-rated Italian restaurants in their vicinity by utilizing ElasticSearch's powerful search capabilities.

8. Order Placement Services

Managing the entire order life cycle, these services provide information to restaurants and update customers on order status. The Order Queue facilitates communication with Restaurant Services, while requests for delivery include

interactions with Delivery Services. Implementing Apache Kafka as a message broker ensures efficient order processing and communication between services.

9. Delivery Services

Facilitating order pickup, these services establish seamless communication between agents and customers, enabling real-time order tracking. They are instrumental in ensuring successful deliveries and collecting customer feedback. For instance, Delivery Services efficiently coordinate the delivery of food from restaurants to customers using real-time tracking tools like Google Maps.

10. Map Services

Searching for nearby agents and tracking orders, Map Services provide real-time location updates to customers. Using technologies like Google Maps, it connects users to the service for order tracking through long polling, minimizing traffic while providing timely updates.

11. Notification Queue

Services wishing to send notifications to users push messages to the Notification Queue, with Notification Services consuming these requests, subsequently sending notifications to users. For example, Kafka can be employed to implement a robust and scalable notification queue.

12. Cache

Storing ongoing requests for quick retrieval, the cache provides speedy access to customers and other microservices. Redis cache, for example, efficiently stores and retrieves frequently accessed data, improving overall system performance.

13. Database

Storing user data, restaurant information, and menus, the database is shared and replicated across various servers. Horizontal sharding, exemplified by MySQL for high consistency in user and transaction data, is complemented by NoSQL for non-critical metadata such as restaurant ratings and delivery agent details. For instance, MongoDB can be used for its flexibility in handling unstructured data like ratings and agent information.

Workflow of the system of Restaurant Management

Below is the step by step process:

- **Step 1:** The customer initiates the registration process through registration services, providing all necessary details and receiving a unique customer ID.
- **Step 2:** Returning customers utilize authentication services for verification.
- **Step 3:** Subsequently, customers explore nearby restaurants using search services, making selections from the CDN-stored restaurant details and menus.
- **Step 4:** Upon making choices, Order Placement Services capture order specifics, encompassing items and quantities. Users then opt for food delivery or dine-in, with the system tracking order status.
- **Step 5:** The Order Placement Services channel the order to the Order Queue, subsequently retrieved by Restaurant Placement Services for acceptance and processing by the restaurant admin.
- **Step 6:** In dine-in scenarios, when the food is ready, Order Placement Services notify customers via the Notification Queue. Notification Services handle the request, sending notifications to the customer.

Below is the step by step process for delivery:

- **Step 1:** Order Placement Services notify the Delivery Queue, prompting Delivery Services to locate nearby delivery agents.
- **Step 2:** Once an agent accepts the order, they connect with the user through Map Services, allowing customers to track

agent locations at regular intervals.

- **Step 3:** Payment transactions are facilitated by Payment Services, redirecting to the respective bank's payment portal or UPI services.
- **Step 4:** Upon successful delivery, the order status is updated to "delivered," with order details stored in the database.
- **Step 5:** Customers can also provide restaurant ratings, saved in a NoSQL database.

Further optimizations for the system

To further enhance the efficiency of the Restaurant Management System, several optimizations can be considered:

- **Caching** strategies, such as implementing a Content Delivery Network (CDN) for static assets like images and menus, can significantly improve response times and reduce server load.
- Employing asynchronous processing for non-time-sensitive tasks, like order notifications, can enhance system responsiveness.
- Efficient database indexing, sharding, and replication strategies should be implemented to ensure fast data retrieval and high availability.
- Load balancing can be optimized dynamically to distribute traffic effectively, and the system should incorporate thorough security measures, including encryption for sensitive data and secure API endpoints.
- Regular performance monitoring and analysis will aid in identifying and addressing potential bottlenecks, ensuring the continued optimization of the Restaurant Management.